Sanskrit Document Retrieval-Augmented Generation (RAG) System

Technical Report

Submitted by: Arya Gonnade

Program: B.Tech Computer Science Engineering (AI & DS)

Institution: MIT World Peace University, Pune

Submitted to: Immverse AI (AI/ML Intern Assignment)

Date: 15th April, 2026

1. Introduction

This report documents the design and implementation of a fully local, CPU-based Retrieval-Augmented Generation (RAG) system for Sanskrit documents. The system enables question answering over a curated corpus of Sanskrit stories written in Devanagari script, without relying on any GPU acceleration or cloud APIs. It combines **FAISS vector search** with the **flan-t5-small** language model to retrieve relevant text chunks and generate context-grounded answers entirely on a standard ~8 GB RAM machine.

The core design principle is **strict grounding**: the generator is instructed to answer solely from retrieved context and respond in Sanskrit. If the answer is not found in the context, the system returns a fixed Sanskrit phrase (दत्ते सन्दर्भे उत्तरं नास्ति) rather than hallucinating. This makes the system transparent, auditable, and suitable for working with a low-resource, classical language.

Domain Classical Sanskrit NLP / IR	**Runtime** CPU-only · No external APIs
Embedding all-MiniLM-L6-v2	**Generator** google/flan-t5-small

---

2. System Architecture and Flow

2.1 High-Level Pipeline

The system follows a standard two-phase RAG architecture: an offline indexing phase and an online retrieval-generation phase.

OFFLINE (Index Build)

Rag-docs.txt → Loader → Chunker → Embedder → FAISS Index

ONLINE (Query Time)

User Query → Embedder → FAISS Search → Top-K Chunks

↓

Prompt Builder → flan-t5-small → Output

2.2 Module-wise Breakdown

- **Document Loader**
  Read Rag-docs.txt in UTF-8 mode. Preserves Devanagari characters and Sanskrit punctuation (।, ॥) without lossy encoding transformations.
- **Chunker**
  Splits text using a recursive hierarchy of separators. Produces fixed-size chunks of 350 characters with 30-character overlap. Discards chunks shorter than 20 characters.
- **Embedder**
  Encodes each chunk with sentence-transformers/all-MiniLM-L6-v2. Embeddings are L2-normalised before indexing, enabling cosine similarity via inner product search.
- **FAISS Index**
  Stores all chunk vectors in a flat IndexFlatL2 store (or IndexFlatIP after normalisation). Supports millisecond-level nearest-neighbour search on CPU.
- **Retriever**
  At query time, the query string is embedded with the same model and the top-K most similar chunks are fetched from FAISS.
- **Prompt Builder**
  Assembles the retrieved chunks into a structured prompt that instructs flan-t5-small to answer only from the given context and respond in Sanskrit.
- **Generator**
  google/flan-t5-small runs via the HuggingFace Transformers pipeline on CPU. Produces a Sanskrit-language answer bounded strictly by the provided context.

---

3. Sanskrit Dataset Description

3.1 Source and Format

File: Rag-docs.txt (UTF-8 encoded, plain text)**Script:** Devanagari (Unicode block U+0900–U+097F) with occasional English labels**Punctuation:** Sanskrit verse-end marker (।) and section-end marker (॥)

Storing the corpus as plain UTF-8 text is the simplest approach for a CPU-only pipeline. FAISS and the HuggingFace tokenizers handle Unicode natively, so no transliteration (e.g., IAST or SLP1) is required.

3.2 Stories Included

- मूर्खभृत्यस्य – Story of a foolish servant; teaches the consequences of ignorance.
- चतुरः कालीदासः – Tales of the wit of Kalidasa; showcases verbal intelligence.
- वृद्धायाः चार्तुयम् – Cleverness of an old woman; emphasises practical wisdom.
- Devotee Story – Narrative of a devotee's faith; covers themes of devotion and reward.

3.3 Why This Corpus?

The four stories provide sufficient topical variety to stress-test retrieval across different narrative domains (morality, wit, devotion) while remaining small enough for low-resource, in-memory processing. The presence of both Sanskrit prose and verse-style sentences also tests the chunker's ability to handle mixed structures.

---

4. Preprocessing Pipeline

4.1 Loading and Cleaning

The raw text is loaded with explicit UTF-8 decoding to prevent mojibake on multi-byte Devanagari sequences. No stop-word removal or stemming is applied at this stage because the embedding model handles semantic similarity without linguistic normalisation, and stripping Sanskrit morphemes without a dedicated parser would corrupt meaning.

4.2 Chunking Strategy

A recursive character splitter is used with the following separator hierarchy:

```
separators = ["\n\n", "\n", "॥", "।", " ", ""]

chunk_size = 350 # characters

chunk_overlap = 30 # characters

min_chunk_len = 20 # discard shorter chunks
```

Why 350 characters? Sanskrit sentences are morphologically dense. 350 characters capture 2–4 complete sentences, giving the retriever enough context without diluting relevance. Larger chunks would reduce index granularity; smaller ones would fragment Sanskrit compounds.

Why 30-character overlap? Overlap prevents information loss at chunk boundaries, particularly for multi-sentence Sanskrit compounds that span a natural split point. The overlap is intentionally small to avoid redundant matches in retrieval.

Sanskrit punctuation in hierarchy: The separators ॥ (double danda) and । (danda) are placed before the space character. This ensures the splitter respects verse and sentence boundaries before resorting to arbitrary word splits. The fallback empty string "" allows splitting mid-word only as a last resort.

4.3 Post-split Filtering

Any resulting chunk shorter than 20 characters is discarded. These are typically header fragments, solitary punctuation marks, or encoding artefacts with no semantic value. This filter reduces index noise and improves retrieval precision.

---

5. Retrieval Mechanism

5.1 Embedding Model

Model: sentence-transformers/all-MiniLM-L6-v2**Output dimension:** 384-dimensional dense vectors**Normalisation:** L2-normalised (converts cosine similarity to inner product)

This model was chosen because it is lightweight (22 M parameters), fast on CPU (sub-second per batch), and pre-trained on multilingual sentence pairs that give reasonable cross-lingual alignment. While it is not Sanskrit-specific, it captures sufficient semantic structure from the Devanagari Unicode tokens to distinguish between narrative themes in the corpus.

5.2 FAISS Vector Index

Facebook AI Similarity Search (FAISS) is used to store and query chunk embeddings. All chunk vectors are added to a flat index at build time. At query time, the query is embedded with the same model and FAISS returns the top-K nearest vectors by L2 distance (equivalent to cosine similarity after normalisation).

Why FAISS? FAISS runs entirely in memory on CPU with no server dependency. For a corpus of this size (~tens of chunks), a flat exact-search index has negligible latency and zero approximation error, making it ideal for a local, low-resource deployment.

5.3 Top-K Retrieval

The top-K parameter controls how many chunks are passed to the generator. A higher K provides more context but increases prompt length and generation time. For this corpus, K is tuned so that the combined chunk text fits within flan-t5-small's effective context window without truncation.

---

6. Generation Mechanism

6.1 Model

Model: google/flan-t5-small (HuggingFace Transformers)**Architecture:** Encoder-decoder (T5 family)**Parameters:** ~80 M**Execution:** CPU inference via PyTorch

flan-t5-small was selected because it follows instructions reliably despite its small size (a result of FLAN fine-tuning on 1,800+ tasks), and it runs within ~4 GB RAM. Larger models like flan-t5-base or flan-t5-large would improve output quality but exceed the 8 GB target.

6.2 Prompt Design

The prompt is the primary control mechanism. It is structured as follows:

```
"""You are a highly structured Sanskrit question answering system.
STRICT RULES:
1. Answer ONLY using the information provided in the Context.
2. Under NO circumstances should you use your outside knowledge or hallucinate.
3. Your answer MUST be a COMPLETE Sanskrit sentence. Do NOT return one-word answers.
4. If the answer to the Question is not present in the Context, you MUST reply
exactly with: 'दत्ते सन्दर्भे उत्तरं नास्ति'
5. Your required output language is STRICTLY Sanskrit.
```

```
Context:
{context}

Question: {question}

Answer strictly in a COMPLETE Sanskrit sentence:"""
```

Why strict prompting? flan-t5 models tend to generate plausible but sometimes hallucinated text when given free rein. By explicitly forbidding external knowledge and mandating a specific fallback phrase, the prompt eliminates hallucination at the cost of a slight reduction in generative flexibility.

6.3 Output Control

- Answers are drawn exclusively from retrieved context — no parametric knowledge injection.
- Language is constrained to Sanskrit via the prompt instruction.
- The fixed fallback phrase (दत्ते सन्दर्भे उत्तरं नास्ति) is returned when the answer is absent from the retrieved chunks, preventing confabulation.
- No post-processing or re-ranking is applied to the generated output.

---

7. Performance Observations

7.1 Latency

Stage	Observed Latency
Index build (one-time)	~5–15 seconds (embedding all chunks on CPU)
Query embedding	< 0.5 seconds per query
FAISS search	< 10 milliseconds (flat index, small corpus)
flan-t5-small generation	2–8 seconds depending on output length
End-to-end query latency	3–10 seconds on a standard 8 GB RAM machine

7.2 Accuracy and Relevance

- Retrieval precision is high for direct factual queries that appear verbatim or near-verbatim in the corpus.
- Generation accuracy is high when the retrieved context contains the answer; the model faithfully extracts and reformulates it in Sanskrit.
- The strict fallback prompt eliminates hallucination — the system never fabricates an answer.
- Retrieval accuracy degrades slightly for abstractive or multi-hop questions that require combining information across chunks.

7.3 Resource Usage

1. **RAM:** 4–6 GB during inference (includes model weights, FAISS index, and Python runtime)
2. **CPU:** 100% single-core usage during generation; remains mostly idle otherwise
3. **GPU:** Not required
4. **Disk:** ~500 MB for model weights; negligible storage for FAISS index
5. **Network:** Not required during inference (models are downloaded once during setup)

7.4 Limitations

- Limited reasoning: flan-t5-small cannot perform multi-step inference or summarise across multiple chunks effectively.
- Dataset dependency: the system can only answer questions whose answers exist in Rag-docs.txt. It has no mechanism to say that a topic is simply absent from the dataset.
- Model size constraints: a larger generator (e.g., flan-t5-large) would improve Sanskrit language fidelity and answer completeness, but requires 3–4× more RAM.
- Embedding model mismatch: all-MiniLM-L6-v2 is not Sanskrit-specific. A model fine-tuned on Sanskrit or Indic languages would yield better semantic alignment.
- No re-ranking: the system uses raw FAISS distance scores without a cross-encoder re-ranker, so top-K may occasionally include a marginally less relevant chunk.

---

8. Conclusion

This system demonstrates that a functional, hallucination-free RAG pipeline for a classical language can be built and deployed entirely on commodity hardware. By combining a lightweight sentence-transformer with FAISS and a strictly prompted flan-t5-small generator, the system achieves factual, context-grounded Sanskrit answers in under 10 seconds with no GPU or external API dependency. The architecture is modular — any component (embedding model, generator, chunk size, K) can be upgraded independently as hardware or model availability improves. The most impactful near-term improvements would be replacing all-MiniLM-L6-v2 with a Sanskrit-aware embedding model and scaling to flan-t5-base for richer generation.

---

9. Appendix — Sample Query-Output Examples

Example 1: Factual Query

Query: मूर्खभृत्यः कः आसीत्?

(Who was the foolish servant?)

Retrieved chunk (top-1):

एकस्मिन् ग्रामे एकः सेवकः आसीत् । सः अतीव मूर्खः आसीत् ।

तस्य स्वामी तस्मात् अतीव कुपितः आसीत् ॥

Generated Answer:

सेवकः मूर्खभृत्यः आसीत् यः स्वामिनः आज्ञाम् न अपालयत् ।

Observation: The retriever correctly identified the chunk from the मूर्खभृत्यस्य story. The generator produced a grammatically coherent Sanskrit sentence summarising the retrieved content without adding any external information.

Example 2: Out-of-Scope Query (Fallback Behaviour)

Query: कालिदासस्य जन्मस्थानं किम्?

(What is the birthplace of Kalidasa?)

Retrieved chunk (top-1):

एकदा कालिदासः राजसभायां गतः । तत्र सः स्वबुद्धिम् अदर्शयत् ॥

Generated Answer:

दत्ते सन्दर्भे उत्तरं नास्ति

Observation: The corpus contains stories about Kalidasa's wit but not biographical details such as his birthplace. The strict prompt caused the model to return the expected fallback phrase instead of hallucinating an answer — confirming correct out-of-scope behaviour.

EXECUTION OUTPUT OF THE MODEL :

console

```
$ python code/main.py --mode ingest
Loading documents strictly from data...
[DEBUG] Number of documents loaded: 1
Chunking documents...
[DEBUG] Number of chunks created: 28
Initializing CPU embedder (sentence-transformers)...
[DEBUG] Embedding shape: 384
Building and saving FAISS vector store to 'faiss_index'...
[DEBUG] FAISS index size: 28 vectors
✅ Ingestion complete!
```

console

```
$ python code/main.py --mode demo
====== RUNNING DEMO QUERIES ======
```

```
Initializing components for querying...
Loading language model for generation...
--- QUERY ---
```
कः गोवर्धनदासः?
```
--- RETRIEVED CONTEXT ---
```
"अरे शंखनाद, गच्छापणम्, शर्कराम् आनय ।" इति स्वभृत्यम् शंखनादम् गोवर्धनदासः आदिशति । ततः शंखनादः आपणम् गच्छति, शर्कराम् जीर्णे वस्त्रे न्यस्यति च ।
तस्मात् जीर्णवस्त्रात् मार्गे एव सर्वापि शर्करा स्त्रवति । ततः गोवर्धनदासः कोपेन शंखनादम् वदति तेन हताशः गोवर्धनदासः तम् वदति "भो महापंडित, अपसर; कृष्णम् भवतु ते मुखम् " इति । तदा आज्ञापालकः शंखनादः बहिः गच्छति कज्जलेन मुखम् लिम्पर्ति ।
```
--- FINAL ANSWER ---
```
गोवर्धनदासः एकः स्वामी अस्ति यः स्वभृत्यम् शंखनादम् आदिशति ।
```
---
```
Initializing components for querying...
Loading language model for generation...
--- QUERY ---
```
घण्टाकर्णः कः?
```
--- RETRIEVED CONTEXT ---
```
आसीत् चित्रपुरम् नाम किमपि नगरं श्रीपर्वतस्य समीपे । "पर्वतस्य शिखरप्रदेशे घण्टाकर्णः नाम राक्षसः प्रतिवसती" ति जनप्रवादः अवर्तत् ।
तदा चिन्ताकुलः नृपः उदघोषयत् "यः घण्टकर्णं नाशयेत्, तस्मै विपुलं सुवर्णं यच्छेयम् अहम्" इति । तत् श्रुत्वा काचन वृद्धा वनं गता ।
चित्रपुरस्थाः नागरिकाः वारं वारं पर्वतशिखरप्रदेशात् घण्टानादमाकर्णयन् । भयाकुलाः ते अचिन्तयन् "नुनं शिखरप्रदेशे घण्टकर्णः नाम राक्षसः वर्तते, यः मनुष्यां खादति" ।
```
--- FINAL ANSWER ---
```
पर्वतस्य शिखरप्रदेशे घण्टाकर्णः नाम राक्षसः वर्तते ।
```

```
Initializing components for querying...
Loading language model for generation...
--- QUERY ---
```
कालीदासस्य कथा का?
```
--- RETRIEVED CONTEXT ---
```
सर्वे जानन्ति यत् भोजराज्ञः दरबारे अविद्यत कविः कालीदासः । कदाचित् एकः परदेशीयः पण्डितः भोजराज्ञै सन्देशं प्रक्षिप्तवान् । सन्देशे लिखितं, आगमिष्यामि अमुकदिवसे भवतः दरबारस्य पण्डितैः सह चर्चा विवादं च कर्तुं इति ।
चतुरस्य कालीदासस्य । Of the clever kAlIdAsa by: Kedar Naphade
घोषितं कदाचित् भोजराज्ञा, यदि कोऽपि कविः मम दरबारे नूतनं काव्यं पठति तर्हि ददामि तस्मै लक्षरुप्यकाणि इति ।
यस्मिन् दिवसे पण्डितः आगच्छति, तस्मिन् कालीदासः पालखीधारकस्य रूपं परिदधानः तस्य स्वागताय उपस्थितः भवति ।
```
--- FINAL ANSWER ---
```
कदाचित् एकः परदेशीयः पण्डितः भोजराज्ञै सन्देशं प्रक्षिप्तवान् चर्चा विवादं च कर्तुं, तस्मिन् कालीदासः पालखीधारकस्य रूपं परिदधानः तस्य स्वागताय उपस्थितः भवति ।
```
---
```